

Quiz: Single Agent Systems & Agent Pipelines

Ques 1: What is a stateful directed graph, and how is it different from a linear pipeline?

Ans: Imagine ordering food at a restaurant where the waiter actually remembers what you ordered earlier and can change your order based on that. That is like a stateful directed graph, it remembers what happened before and can take different paths depending on that memory. Now compare that to a vending machine. You press a button, it gives you the item, and it forgets you the second you walk away. It always follows the same fixed steps no matter what. That vending machine is like a simple linear pipeline, it just goes step by step with no memory and no way to branch off.

Ques 2: What are nodes and edges? Give an example of each.

Ans: Think about a customer support chatbot for a phone company. One task the bot can do is "check my bill," and that whole task is a node. Another task might be "reset my password," also a node. Now, the connection that decides the bot moves from reading your message to actually running the "check my bill" task, that connecting arrow is the edge. So nodes are the actual jobs getting done, and edges are just the paths that tell the system where to go next.

Ques 3: What is conditional routing? Design a rule based routing logic for three query types.

Ans: Conditional routing simply means the system reads your message and figures out where to send it based on what you are asking. Let us say I am building a chatbot for a phone company. If the message contains the word "bill" or "payment," I send it to the Billing Handler. If the message contains the word "not working" or "broken," I send it to the Technical Support Handler. If it does not match either of those, I just send it to a General Help Handler that gives a normal reply.

Ques 4 Why are loops important in agent pipelines? Give a use case where a retry loop is necessary.

Ans: Loops let the system try something again instead of just quitting after one failed attempt. Picture uploading a photo to social media on a train with spotty wifi. The upload fails because the connection dropped for a second, not because anything is actually wrong with your photo. A retry loop lets the app quietly try

uploading again two or three times before finally telling you it failed. Without that loop, it would just show an error the first time your wifi hiccups, which would be really annoying.

Ques 5: Explain how a single agent system can simulate multi agent behavior internally.

Ans: Think of someone cooking dinner completely alone. First they act like a shopper, going through the fridge to see what ingredients are available. Then they switch into the role of a chef, actually cooking the meal. Then they switch again into the role of a waiter, plating the food and bringing it to the table. It is still just one person the whole time, but they are mentally switching between different roles one after another. A single agent system works the same way, one program plays multiple roles in sequence instead of needing several separate agents.

Ques 6: What are JSON schema tools? How do they help in structuring tool inputs and outputs?

Ans: Think of signing up for a newsletter online. The form forces you to type your email in an actual email format and won't let you submit your name in the phone number box. That form is basically acting like a JSON schema, it sets strict rules for what kind of data goes where before anything gets accepted. In an agent system, a JSON schema does the same job, it makes sure the data going into a tool and coming out of a tool always follows the same expected shape, so nothing weird or broken gets passed around.

Ques 7: Compare sequential and parallel tool calls. When would you prefer one over the other?

Ans: Baking a cake is sequential, you have to mix the batter before you can put it in the oven, there is no way to skip that order. But washing your clothes and washing your dishes at the same time is parallel, one task has nothing to do with the other, so you can run both machines at once and save time. In an agent system, you use sequential calls when one step actually needs the result of the step before it, and you use parallel calls when the tasks are totally unrelated and doing them together just saves time.

Ques 8: How would you implement error handling in a tool using agent? Provide at least two strategies.

Ans: Think about using an ATM. First strategy is like the ATM catching a problem calmly, if your card is inserted the wrong way, it tells you clearly

"please reinsert your card" instead of just freezing or breaking. That is basically a try and catch approach, catching the problem and giving a clear message. Second strategy is like the ATM automatically trying to read your card two or three times before finally saying it cannot read it, that is a retry approach. Both of these keep the whole system from just crashing the moment something small goes wrong.

Ques 9: What is trajectory evaluation? Why is it important beyond checking the final output?

Ans: Think about a teacher grading a math exam. If the teacher only looked at your final answer, a student could have made two mistakes that accidentally canceled out and still landed on the right number. But a teacher who checks every step of your working out can catch that you actually made mistakes along the way, even though the final answer happened to be correct. Trajectory evaluation in an agent works the same way, it looks at every decision and every tool call the agent made, not just whether the last answer looked fine, because that helps catch hidden problems in how the agent actually thinks.

Ques 10: Define task completion rate and cost metrics. How would you measure and optimize them in a real world system?

Ans: Picture a food delivery app. Task completion rate would be how many orders actually got delivered successfully out of all the orders that came in. Cost metrics would be things like how much fuel or time it took the delivery drivers on average per order. You measure completion rate by simply tracking how many deliveries succeeded versus failed over time, and you measure cost by tracking things like time spent or resources used per task. To improve both, you would want to fix whatever is causing failed deliveries, like bad routing or wrong addresses, and also find ways to cut down wasted time or resources per delivery without hurting how many actually get completed.